

INSTITUTO TECNOLÓGICO SUPERIOR DEL ORIENTE DEL ESTADO DE HIDALGO

Ingeniería en Sistemas Computacionales

Docente:
Efren Rolando Romero Leon.

Asignatura:
Métodos numéricos.

Evidencia:
Problemario.

Grupo:
4F21

Integrantes del equipo:
Hernández León Luis Angel.
Carrasco Escalante Felipe.
Romero Pérez Esteban.
Urdanivia Campos Miguel Angel.
de la Rosa Sánchez Saray.

Periodo:
Enero - Junio 2026

Eliminación Gaussiana.

Código en Python.

Es un método directo que utiliza operaciones entre filas para transformar la matriz del sistema en una matriz triangular superior. Una vez hecho esto, se encuentran las incógnitas mediante un proceso llamado "sustitución hacia atrás".

Una de sus ventajas es que es un método muy sistemático y garantiza encontrar la solución exacta (salvo errores de redondeo de la computadora). Es la base de casi todos los métodos directos.

Ejercicio 1)

```
=====
ELIMINACION GAUSSIANA – MENU PRINCIPAL
=====

Que quieres hacer?
[1] Ejemplo 1 – Sistema 3x3 (solucion unica)
[2] Ejemplo 2 – Sistema 2x2 (incompatible)
[3] Ingresar mi propio sistema
[0] Salir

Opcion: 3
```

```
=====
INGRESA TU PROPIO SISTEMA
=====

cuantas ecuaciones tiene tu sistema? (ej: 2, 3, 4): 4

Ingresa los coeficientes separados por espacios.
Ejemplo: para 2x + y - z = 8 escribe: 2 1 -1

Ecuacion 1 - coeficientes [x, y, z, w]: 6 7 4
Necesitas exactamente 4 coeficientes.
Ecuacion 1 - coeficientes [x, y, z, w]: 7 8 9 5
Ecuacion 2 - coeficientes [x, y, z, w]: 5 4 3 6
Ecuacion 3 - coeficientes [x, y, z, w]: 7 5 4 8
Ecuacion 4 - coeficientes [x, y, z, w]: 7 5 3 2
```

Ahora los terminos independientes (lado derecho del =).

Terminos independientes [4 valores]: 3 6 1 1

Sistema ingresado:

$$(7)x + (8)y + (9)z + (5)w = 3$$

$$(5)x + (4)y + (3)z + (6)w = 6$$

$$(7)x + (5)y + (4)z + (8)w = 1$$

$$(7)x + (5)y + (3)z + (2)w = 1$$

=====

ELIMINACIÓN GAUSSIANA CON PIVOTEO PARCIAL

=====

Matriz aumentada inicial [A|b]:

$$\begin{bmatrix} 7.0000 & 8.0000 & 9.0000 & 5.0000 & | & 3.0000 \\ 5.0000 & 4.0000 & 3.0000 & 6.0000 & | & 6.0000 \\ 7.0000 & 5.0000 & 4.0000 & 8.0000 & | & 1.0000 \\ 7.0000 & 5.0000 & 3.0000 & 2.0000 & | & 1.0000 \end{bmatrix}$$

Pivote en columna 1: 7.0000

$$m_{21} = 0.7143 \rightarrow F_2 = F_2 - (0.7143) \times F_1$$

$$m_{31} = 1.0000 \rightarrow F_3 = F_3 - (1.0000) \times F_1$$

$$m_{41} = 1.0000 \rightarrow F_4 = F_4 - (1.0000) \times F_1$$

Despues de eliminar columna 1:

$$\begin{bmatrix} 7.0000 & 8.0000 & 9.0000 & 5.0000 & | & 3.0000 \\ 0.0000 & -1.7143 & -3.4286 & 2.4286 & | & 3.8571 \\ 0.0000 & -3.0000 & -5.0000 & 3.0000 & | & -2.0000 \\ 0.0000 & -3.0000 & -6.0000 & -3.0000 & | & -2.0000 \end{bmatrix}$$

Pivoteo: intercambiamos F2 con F3

$$\begin{bmatrix} 7.0000 & 8.0000 & 9.0000 & 5.0000 & | & 3.0000 \\ 0.0000 & -3.0000 & -5.0000 & 3.0000 & | & -2.0000 \\ 0.0000 & -1.7143 & -3.4286 & 2.4286 & | & 3.8571 \\ 0.0000 & -3.0000 & -6.0000 & -3.0000 & | & -2.0000 \end{bmatrix}$$

Pivote en columna 2: -3.0000

$$m_{32} = 0.5714 \rightarrow F_3 = F_3 - (0.5714) \times F_2$$

$$m_{42} = 1.0000 \rightarrow F_4 = F_4 - (1.0000) \times F_2$$

Despues de eliminar columna 2:

[	7.0000	8.0000	9.0000	5.0000		3.0000	]
[	0.0000	-3.0000	-5.0000	3.0000		-2.0000	]
[	0.0000	0.0000	-0.5714	0.7143		5.0000	]
[	0.0000	0.0000	-1.0000	-6.0000		0.0000	]

Pivoteo: intercambiamos F3 con F4

[	7.0000	8.0000	9.0000	5.0000		3.0000	]
[	0.0000	-3.0000	-5.0000	3.0000		-2.0000	]
[	0.0000	0.0000	-1.0000	-6.0000		0.0000	]
[	0.0000	0.0000	-0.5714	0.7143		5.0000	]

Pivote en columna 3: -1.0000

$m_{43} = 0.5714 \rightarrow F_4 = F_4 - (0.5714) \times F_3$

Despues de eliminar columna 3:

[	7.0000	8.0000	9.0000	5.0000		3.0000	]
[	0.0000	-3.0000	-5.0000	3.0000		-2.0000	]
[	0.0000	0.0000	-1.0000	-6.0000		0.0000	]
[	0.0000	0.0000	0.0000	4.1429		5.0000	]

Pivote en columna 4: 4.1429

Despues de eliminar columna 4:

[	7.0000	8.0000	9.0000	5.0000		3.0000	]
[	0.0000	-3.0000	-5.0000	3.0000		-2.0000	]
[	0.0000	0.0000	-1.0000	-6.0000		0.0000	]
[	0.0000	0.0000	0.0000	4.1429		5.0000	]

Sustitucion hacia atras:

$$\begin{aligned}x_4 &= (5.0000 - 0.0000) / 4.1429 = 1.2069 \\x_3 &= (0.0000 - -7.2414) / -1.0000 = -7.2414 \\x_2 &= (-2.0000 - 39.8276) / -3.0000 = 13.9425 \\x_1 &= (3.0000 - 52.4023) / 7.0000 = -7.0575\end{aligned}$$

SOLUCION:

$$\begin{aligned}x &= -7.0575 \\y &= 13.9425 \\z &= -7.2414 \\w &= 1.2069\end{aligned}$$

Verificacion:

$$\begin{aligned}\text{Ecuacion 1: } 3.0000 &= 3.0 \quad [\text{OK}] \\ \text{Ecuacion 2: } 6.0000 &= 6.0 \quad [\text{OK}] \\ \text{Ecuacion 3: } 1.0000 &= 1.0 \quad [\text{OK}] \\ \text{Ecuacion 4: } 1.0000 &= 1.0 \quad [\text{OK}]\end{aligned}$$

Ejercicio 2)

```
=====
      INGRESA TU PROPIO SISTEMA
=====

cuantas ecuaciones tiene tu sistema? (ej: 2, 3, 4): 3

Ingresa los coeficientes separados por espacios.
Ejemplo: para 2x + y - z = 8 escribe: 2 1 -1

Ecuacion 1 - coeficientes [x, y, z, w]: 400 -500 398 234
Necesitas exactamente 3 coeficientes.
Ecuacion 1 - coeficientes [x, y, z, w]: -233 432 424
Ecuacion 2 - coeficientes [x, y, z, w]: 244 -487 -342
Ecuacion 3 - coeficientes [x, y, z, w]: 248 392 492

Ahora los terminos independientes (lado derecho del =).

Terminos independientes [3 valores]: 340 231 342
```

Sistema ingresado:

$$\begin{aligned}(-233)x + (432)y + (424)z &= 340 \\(244)x + (-487)y + (-342)z &= 231 \\(248)x + (392)y + (492)z &= 342\end{aligned}$$

=====

ELIMINACIÓN GAUSSIANA CON PIVOTEO PARCIAL

=====

Matriz aumentada inicial $[A|b]$:

$$\begin{bmatrix} -233.0000 & 432.0000 & 424.0000 & | & 340.0000 \\ 244.0000 & -487.0000 & -342.0000 & | & 231.0000 \\ 248.0000 & 392.0000 & 492.0000 & | & 342.0000 \end{bmatrix}$$

Pivoteo: intercambiamos F1 con F3

$$\begin{bmatrix} 248.0000 & 392.0000 & 492.0000 & | & 342.0000 \\ 244.0000 & -487.0000 & -342.0000 & | & 231.0000 \\ -233.0000 & 432.0000 & 424.0000 & | & 340.0000 \end{bmatrix}$$

Pivote en columna 1: 248.0000

$$\begin{aligned}m_{21} &= 0.9839 \rightarrow F_2 = F_2 - (0.9839) \times F_1 \\m_{31} &= -0.9395 \rightarrow F_3 = F_3 - (-0.9395) \times F_1\end{aligned}$$

Despues de eliminar columna 1:

$$\begin{bmatrix} 248.0000 & 392.0000 & 492.0000 & | & 342.0000 \\ 0.0000 & -872.6774 & -826.0645 & | & -105.4839 \\ 0.0000 & 800.2903 & 886.2419 & | & 661.3145 \end{bmatrix}$$

Pivote en columna 2: -872.6774

$$m_{32} = -0.9171 \rightarrow F_3 = F_3 - (-0.9171) \times F_2$$

Despues de eliminar columna 2:

$$\begin{bmatrix} 248.0000 & 392.0000 & 492.0000 & | & 342.0000 \\ 0.0000 & -872.6774 & -826.0645 & | & -105.4839 \\ 0.0000 & 0.0000 & 128.6981 & | & 564.5804 \end{bmatrix}$$

Pivote en columna 3: 128.6981

Despues de eliminar columna 3:

$$\begin{bmatrix} 248.0000 & 392.0000 & 492.0000 & | & 342.0000 \\ 0.0000 & -872.6774 & -826.0645 & | & -105.4839 \\ 0.0000 & 0.0000 & 128.6981 & | & 564.5804 \end{bmatrix}$$

Sustitucion hacia atras:

$$\begin{aligned}x_3 &= (564.5804 - 0.0000) / 128.6981 = 4.3869 \\x_2 &= (-105.4839 - -3623.8294) / -872.6774 = -4.0317 \\x_1 &= (342.0000 - 577.9213) / 248.0000 = -0.9513\end{aligned}$$

SOLUCION:

$$\begin{aligned}x &= -0.9513 \\y &= -4.0317 \\z &= 4.3869\end{aligned}$$

Verificacion:

Ecuacion 1: $340.0000 = 340.0$ [OK]
Ecuacion 2: $231.0000 = 231.0$ [OK]
Ecuacion 3: $342.0000 = 342.0$ [OK]

Ejercicio 3)

Que quieres hacer?

- [1] Ejemplo 1 – Sistema 3x3 (solucion unica)
- [2] Ejemplo 2 – Sistema 2x2 (incompatible)
- [3] Ingresar mi propio sistema
- [0] Salir

Opcion: 2

ELIMINACIÓN GAUSSIANA CON PIVOTEO PARCIAL

Matriz aumentada inicial $[A|b]$:

1.0000	1.0000	2.0000
2.0000	2.0000	5.0000

Pivoteo: intercambiamos F1 con F2

2.0000	2.0000	5.0000
1.0000	1.0000	2.0000

Pivote en columna 1: 2.0000

$m_{21} = 0.5000 \rightarrow F2 = F2 - (0.5000) \times F1$

Despues de eliminar columna 1:

2.0000	2.0000	5.0000
0.0000	0.0000	-0.5000

SISTEMA INCOMPATIBLE

Fila 2: $0 = -0.5000 \rightarrow$ Contradiccion

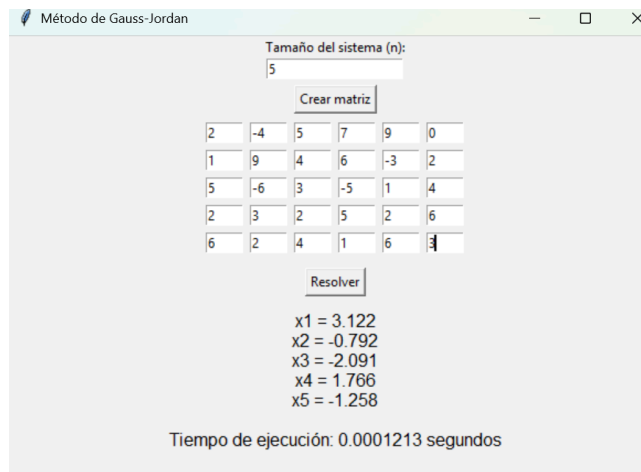
El sistema no tiene solucion.

Método de Gauss - Jordan

Código en Python

El método de Gauss - Jordan es una extensión de la Eliminación Gaussiana. En lugar de detenerse en una matriz triangular superior, continúa haciendo ceros también por encima de la diagonal principal, convirtiendo la matriz original en una matriz identidad. Una ventaja de este método es que elimina la necesidad de hacer la sustitución hacia atrás, ya que los resultados quedan directamente en la columna de términos independientes. Es excelente para calcular la matriz inversa.

Ejercicio 1)



Tamaño del sistema (n):
5

Crear matriz

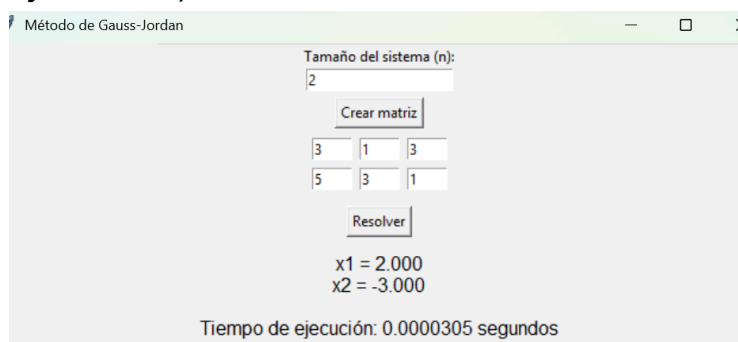
2	-4	5	7	9	0
1	9	4	6	-3	2
5	-6	3	-5	1	4
2	3	2	5	2	6
6	2	4	1	6	3

Resolver

$x_1 = 3.122$
 $x_2 = -0.792$
 $x_3 = -2.091$
 $x_4 = 1.766$
 $x_5 = -1.258$

Tiempo de ejecución: 0.0001213 segundos

Ejercicio 2)



Tamaño del sistema (n):
2

Crear matriz

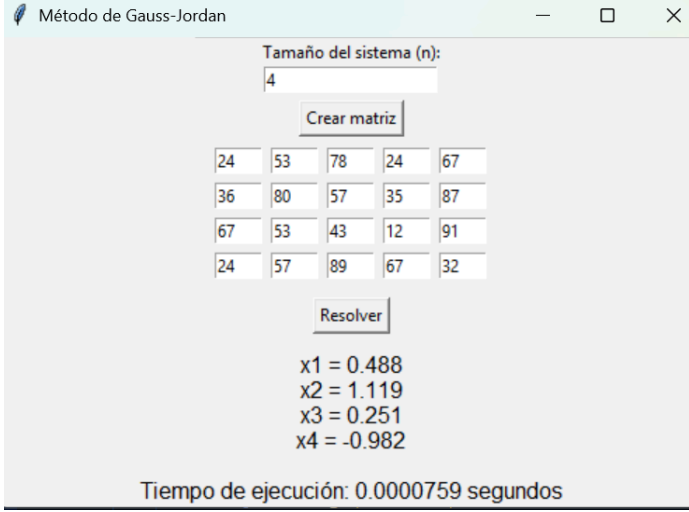
3	1	3
5	3	1

Resolver

$x_1 = 2.000$
 $x_2 = -3.000$

Tiempo de ejecución: 0.0000305 segundos

Ejercicio 3)



Método de Gauss-Jordan

Tamaño del sistema (n):
4

Crear matriz

24	53	78	24	67
36	80	57	35	87
67	53	43	12	91
24	57	89	67	32

Resolver

x1 = 0.488
x2 = 1.119
x3 = 0.251
x4 = -0.982

Tiempo de ejecución: 0.0000759 segundos

Método de Gauss-Seidel

Es una mejora del método de Jacobi. La diferencia clave es que, a medida que calcula el nuevo valor de una variable, lo utiliza inmediatamente para calcular la siguiente variable dentro de la misma iteración.

Ventajas: Por lo general, converge a la solución mucho más rápido que Jacobi, lo que significa que requiere menos iteraciones y menos tiempo de cómputo para llegar a la misma respuesta.

Ejercicio 1)

```
def gauss_seidel(A, b, tolerance=1e-5, max_iterations=100):  
    """  
    Resuelve el sistema  $Ax = b$  usando el método de Gauss-Seidel.  
    """  
    n = len(b)  
    # Vector inicial de ceros  
    x = np.zeros_like(b, dtype=np.double)  
  
    for k in range(max_iterations):  
        x_old = x.copy()  
  
        for i in range(n):  
            # Suma de la parte ya actualizada (j < i)  
            sum1 = sum(A[i][j] * x[j] for j in range(i))  
            # Suma de la parte de la iteración anterior (j > i)  
            sum2 = sum(A[i][j] * x_old[j] for j in range(i + 1, n))  
  
            # Cálculo del nuevo valor para x_i  
            x[i] = (b[i] - sum1 - sum2) / A[i][i]  
  
        # Calcular el error (norma euclidiana)  
        error = np.linalg.norm(x - x_old, ord=np.inf)  
  
        # Verificar convergencia  
        if error < tolerance:  
            print(f"Convergió en {k+1} iteraciones.")  
            return x  
  
        print("El método no convergió en el número máximo de iteraciones.")  
        return x  
  
# --- Ejemplo de uso ---  
A = np.array([[5, -1, 1],  
              [2, 8, -1],  
              [-1, 1, 4]], dtype=float)  
b = np.array([1, 2, 3], dtype=float)  
  
solucion = gauss_seidel(A, b)  
print("Solución:", solucion)
```

Convergió en 13 iteraciones.
Solución: [2.50000535 1.37179712 2.97436172]

Ejercicio 2)

```
def gauss_seidel(A, b, tolerance=1e-5, max_iterations=100):  
    """  
    Resuelve el sistema  $Ax = b$  usando el método de Gauss-Seidel.  
    """  
    n = len(b)  
    # Vector inicial de ceros  
    x = np.zeros_like(b, dtype=np.double)  
  
    for k in range(max_iterations):  
        x_old = x.copy()  
  
        for i in range(n):  
            # Suma de la parte ya actualizada (j < i)  
            sum1 = sum(A[i][j] * x[j] for j in range(i))  
            # Suma de la parte de la iteración anterior (j > i)  
            sum2 = sum(A[i][j] * x_old[j] for j in range(i + 1, n))  
  
            # Cálculo del nuevo valor para x_i  
            x[i] = (b[i] - sum1 - sum2) / A[i][i]  
  
        # Calcular el error (norma euclidiana)  
        error = np.linalg.norm(x - x_old, ord=np.inf)  
  
        # Verificar convergencia  
        if error < tolerance:  
            print(f"Convergió en {k+1} iteraciones.")  
            return x  
  
        print("El método no convergió en el número máximo de iteraciones.")  
        return x  
  
# --- Ejemplo de uso ---  
A = np.array([[7, 1, -5],  
              [5, 9, -6],  
              [-5, 7, 4]], dtype=float)  
b = np.array([4, 7, 9], dtype=float)  
  
solucion = gauss_seidel(A, b)  
print("Solución:", solucion)
```

Convergió en 8 iteraciones.
Solución: [0.1208782 0.30769278 0.70329636]

Ejercicio 3)

```
def gauss_seidel(A, b, tolerance=1e-5, max_iterations=100):  
    """  
    Resuelve el sistema  $Ax = b$  usando el método de Gauss-Seidel.  
    """  
    n = len(b)  
    # Vector inicial de ceros  
    x = np.zeros_like(b, dtype=np.double)  
  
    for k in range(max_iterations):  
        x_old = x.copy()  
  
        for i in range(n):  
            # Suma de la parte ya actualizada (j < i)  
            sum1 = sum(A[i][j] * x[j] for j in range(i))  
            # Suma de la parte de la iteración anterior (j > i)  
            sum2 = sum(A[i][j] * x_old[j] for j in range(i + 1, n))  
  
            # Cálculo del nuevo valor para x_i  
            x[i] = (b[i] - sum1 - sum2) / A[i][i]  
  
        # Calcular el error (norma euclidiana)  
        error = np.linalg.norm(x - x_old, ord=np.inf)  
  
        # Verificar convergencia  
        if error < tolerance:  
            print(f"Convergió en {k+1} iteraciones.")  
            return x  
  
        print("El método no convergió en el número máximo de iteraciones.")  
        return x  
  
# --- Ejemplo de uso ---  
A = np.array([[9, -1, -5],  
              [-3, 10, 7],  
              [8, 7, -3]], dtype=float)  
b = np.array([1, 5, 2], dtype=float)  
  
solucion = gauss_seidel(A, b)  
print("Solución:", solucion)
```

```
Convergió en 16 iteraciones.  
Solución: [ 0.72331569 -0.06323648  1.11462338]
```

Método de Jacobi

Es un método iterativo. Comienza con una suposición inicial (valores iniciales) para todas las variables y utiliza las ecuaciones para calcular una nueva aproximación. En cada iteración, usa exclusivamente los valores de la iteración anterior para encontrar los nuevos. El proceso se repite hasta que el error es menor a una tolerancia definida.

Ventajas: Es extremadamente fácil de paralelizar en computación porque el cálculo de cada variable es independiente de las demás en la misma iteración.

Ejercicio 1

Método de Jacobi

Número de ecuaciones:

Crear Matriz

Fila 1	4	1
Fila 2	2	5

Vector b (separado por espacios):

Valores iniciales x0:

Tolerancia:

Máximo de iteraciones:

Ejecutar Jacobi

```
Iteración 3: [2.025, 1.08]    Error: 0.22499999999999987
Iteración 4: [1.98, 0.99]    Error: 0.090000000000000008
Iteración 5: [2.0025, 1.008]  Error: 0.022499999999999964
Iteración 6: [1.998, 0.999]   Error: 0.009000000000000008
Iteración 7: [2.00025, 1.0008] Error: 0.0022499999999998632
Iteración 8: [1.9998, 0.9999]  Error: 0.0008999999999999009
Iteración 9: [2.000025, 1.00008] Error: 0.0002249999999999197
Iteración 10: [1.9999799999999999, 0.999999] Error: 9.000000000000345e-05

☒ Convergió en 10 iteraciones

@ Solución aproximada: [1.9999799999999999, 0.999999]
```

Ejercicios 2

Método de Jacobi

Número de ecuaciones:

Crear Matriz

Fila 1	5	-1	1
Fila 2	2	8	-1
Fila 3	-1	1	4

Vector b (separado por espacios):

Valores iniciales x0:

Tolerancia:

Máximo de iteraciones:

Ejecutar Jacobi

```

Iteración 6: [1.0003916015625, 1.000166015625, 0.999422607421875] Error: 0.0013681640625
000657
Iteración 7: [1.0001486816406249, 0.9998299255371094, 1.000056396484375] Error: 0.000633
7890624998899
Iteración 8: [0.9999547058105469, 0.9999698791503906, 1.000079689025879] Error: 0.000193
9758300779637
Iteración 9: [0.9999780380249023, 1.000021284675598, 0.9999962066650391] Error: 8.348236
083988159e-05

☒ Convergió en 9 iteraciones
🔊 Solución aproximada: [0.9999780380249023, 1.000021284675598, 0.9999962066650391]
```

Ejercicio 3

Método de Jacobi

Número de ecuaciones:

Crear Matriz

Fila 1	10	2	-1
Fila 2	-3	-6	2
Fila 3	1	1	5

Vector b (separado por espacios):

Valores iniciales x0:

Tolerancia:

Máximo de iteraciones:

Ejecutar Jacobi

```

Iteración 7: [1.9997307407407408, -0.9997988148148149, 3.000159140740741] Error: 0.00165
02518518519693
Iteración 8: [1.9999756770370372, -0.99981232345679, 3.0000136148148147] Error: 0.000244
93629629640523
Iteración 9: [1.9999638261728392, -0.9999833002469137, 2.9999673292839506] Error: 0.0001
7097679012367806
Iteración 10: [1.9999933929777778, -0.9999928033251027, 3.0000038948148147] Error: 3.656
5530864063334e-05

☒ Convergió en 10 iteraciones
🔊 Solución aproximada: [1.9999933929777778, -0.9999928033251027, 3.0000038948148147]
```

¿Cuál es ideal para sistemas cortos?

Los métodos directos como lo son la eliminación Gaussiana y Gauss - Jordan, ya que estos métodos encuentran la solución exacta de forma rápida sin problemas de tolerancia.

¿Cuál es ideal para sistemas largos?

El método de Gauss-Seidel requiere menos memoria y, si la matriz tiene las propiedades adecuadas (como ser diagonalmente dominante), llegará a una aproximación excelente muy rápido.

¿Qué método tiene el algoritmo más sencillo?

Si se observa desde el modo iterativo el método de Jacobi tiene la lógica de programación más sencilla, ya que los valores no se actualizan en cascada durante la iteración, solo necesitas calcular los nuevos valores en un vector temporal y luego reemplazar el viejo por el nuevo.

Por otro lado el método de eliminación Gaussiana es a menudo la más intuitiva de aprender a mano o codificar inicialmente, ya que es exactamente el mismo proceso de despeje y eliminación que nos enseñan en álgebra básica, paso a paso.